# AAM O82_2
# ROCA-D
# Spécification formelle et contrat d'implémentation

Spécification de laboratoire issue du corpus AAM O81/O82

6 octobre 2026

# Table des matières

**1 Principes directeurs** **5**

**2 Univers formel et contrats** **6**
2.1 Contrat de décision . . . 6
2.2 Contexte d'observation . . . 6
2.3 Épisode et état . . . 6
2.4 Sondes et résultat d'observation . . . 7
2.5 Empreinte partielle . . . 7
2.6 Poset des vues et témoins de raffinement . . . 7
2.7 Équivalence décisionnelle . . . 7

**3 Résultats de vérification et frontière de vérité** **8**
3.1 Interface de vérification . . . 8
3.2 Statuts de compétence et résultat . . . 8
3.3 Suffisance décisionnelle . . . 8

**4 Architecture ROCA-D** **9**
4.1 Vue d'ensemble . . . 9
4.2 Noyaux autoritaires . . . 9

**5 Observation dégradée** **10**
5.1 Principe de moindre effort observable . . . 10
5.2 Boucle de raffinement . . . 10
5.3 Dégradation et continuité . . . 10

**6 Compatibilité et extension du socle O81** **11**
6.1 État runtime canonique . . . 11
6.2 Enveloppe d'action compatible . . . 11
6.3 Instruments spécialistes à interface variable . . . 11
6.4 Self-health . . . 11
6.5 Lifecycle des compétences . . . 12
6.6 Observation progressive et re-observation . . . 12
6.7 Non-régression de potentiel . . . 12
6.8 Règle de contrôle . . . 12
6.9 Valeur marginale normative . . . 12
6.10 Budget vectoriel . . . 13
6.11 Arrêt minimal . . . 13
6.12 Mémoire consultée à chaque pas . . . 13

**7 Mémoire comme géométrie de l'empreinte** **14**
7.1 Noeud de quotient . . . 14
7.2 Correction . . . 14
7.3 Deux tempos et compression . . . 14
7.4 Invalider et oublier . . . 14

**8 Moteur de candidats et de suffisance** **15**
8.1 Schéma d'un candidat . . . 15
8.2 Génération de candidats . . . 15
8.3 Compatibilité . . . 15
8.4 Fermeture et abstention . . . 15

**9 Apprentissage par erreur et correction** **16**
9.1 Cycle fondamental . . . 16
9.2 CEGIS . . . 16
9.3 Primitives et composition . . . 16

**10 Capsules de compétence et acquisition** **17**
10.1 Schema de capsule . . . 17
10.2 Cycle d'admission . . . 17
10.3 Teacher cutover . . . 17

**11 Auto-observation et apprentissage de l'apprentissage** **18**
11.1 État auto-observable . . . 18
11.2 Déclencheurs causaux . . . 18
11.3 Intervention causale . . . 18
11.4 Prediction précoce . . . 18

**12 Instruments à interface variable** **19**
12.1 Contrat spécialiste . . . 19
12.2 Désaccord . . . 19
12.3 Loss, attention, optimiseur . . . 19

**13 Interfaces externes, sécurité et protocoles** **20**
13.1 RAG comme observateur externe . . . 20
13.2 Frontière d'accès . . . 20
13.3 Adaptateurs black-box . . . 20
13.4 Mesure vectorielle de puissance . . . 21
13.5 Protocoles expérimentaux P1-P10 . . . 21

**14 Découverte de sondes** **22**
14.1 Apprentissage textuel généralisant et priorité des preuves . . . 22
14.2 Probe candidate . . . 22

**15 Décomposition, composition et root retest** **23**
15.1 Décomposition . . . 23
15.2 Root retest . . . 23
15.3 Composition de compétences . . . 23

**16 État machine autoritaire** **24**

**17 Invariants d'implémentation** **25**

**18 Protocoles de validation obligatoires** **26**

**19 Contrat CLI et guide PC** **27**
19.1 Installation normative . . . 27
19.2 Commandes normatives . . . 27
19.3 Configuration . . . 27
19.4 Mode FINAL . . . 28
19.5 Mode EXPERIMENT . . . 28
19.6 Recovery . . . 28

**20 Rapports et journalisation** **29**

**21 Erreurs à éviter absolument** **30**

**22 Optimisation rigoureuse** **31**
22.1 Mesure coût-capacité . . . 31
22.2 Allocation adaptative . . . 31
22.3 Portfolios . . . 31
22.4 Pruning . . . 31

**23 Extensions** **32**

**24 Checklist de release normative** **33**

**25 Clôture expérimentale O82_2 / O82_c1** **34**
25.1 Statut expérimental . . . 34
25.2 Portfolio de généralisation . . . 34
25.3 Apprentissage par répétition et correction . . . 34
25.4 Réponse généralisée et confiance graduée . . . 34
25.5 Observation dégradée . . . 35
25.6 Auto-observation causale . . . 35
25.7 Restauration de potentiel O81 . . . 35
25.8 Langage et sortie . . . 35
25.9 Règles de non-régression expérimentale . . . 35
25.10Conclusion de version . . . 35

**26 Conclusion formelle** **37**

# Statut, portée et règle de non-triche

Le présent document est le contrat normatif de l'architecture AAM O82, retenue par le laboratoire sous le nom ROCA-D (*Recursive Observation-Correction Architecture with Degraded Observation*). Il formalise les objets, interfaces, invariants, transitions, protocoles d'acquisition et critères de release nécessaires à une implémentation exécutable sur PC.

Le noyau ne contient aucune réponse métier, aucune règle de domaine cachée, aucun label de tâche futur et aucun paramètre copié d'un modèle externe. Une compétence n'est admissible qu'après acquisition, validation, provenance et vérification selon les contrats de ce document. Les teacher, micro-modèles, RAG ou verificateurs externes sont des adaptateurs facultatifs d'acquisition ou de vérification et ne deviennent jamais une vérité implicite.

Une affirmation d'impossibilité globale est interdite par le modèle. Un état UNRESOLVED signifie uniquement que, sous le contrat, la visibilité, les ressources et les verificateurs effectivement disponibles à cet instant, aucune fermeture certifiable n'a été obtenue. Cette distinction est normative.

# Chapitre 1

# Principes directeurs

1. **Aucune impossibilité pré-déclarée.** Toute limite rapportée est un fait local lié à un budget, une visibilité, un verifier, un support de données ou une expérience explicitement enregistrée.
2. **Observation comme ressource.** Une observation possède un contrat, un coût, une provenance, une visibilité et une sémantique de vue.
3. **Observation dégradée.** L'effort minimal admissible est tenté en premier. L'effort augmente uniquement lorsqu'une preuve de suffisance manque, qu'une correction est requise ou qu'une exploration est justifiée.
4. **Mémoire après observation.** Après chaque observation réussie, le retrieval de mémoire est obligatoire avant toute observation supplémentaire, sauf impossibilité d'intégrité explicitement journaliée.
5. **Erreur exploitable mais non certifiante.** Une erreur est une trace de contre-exemple et peut guider l'action suivante ; elle ne peut pas certifier une réponse.
6. **Suffisance décisionnelle.** Le modèle vise la fermeture du contrat de décision courant, et non l'identification exhaustive du monde.
7. **Vérification séparée.** Hypothèse, learner, politique et verifier sont des rôles distincts ; une politique ne s'auto-certifie pas.
8. **Gouvernance lexicographique.** Sécurité et intégrité sont des filtres durs ; aucune optimisation d'efficacité ne peut les racheter. Les utilités numériques ne sont utilisées qu'à l'intérieur d'une priorité déjà admissible.
9. **Autonomie runtime.** Le runtime FINAL doit fonctionner sans teacher si aucune dépendance de vérification externe n'est déclarée et requise par le contrat ; dans ce dernier cas l'état de sortie doit le signaler.
10. **Auto-observation causale.** L'auto-observation est une intervention mesurée, pas seulement une journalisation.

# Chapitre 2

# Univers formel et contrats

## 2.1 Contrat de décision

Un contrat est

$$g = (id, version, objective, scope, decision_space, required_level, \\ budget_policy, probe_allowlist, security_policy, \\ termination_policy, visibility_policy).$$

*objective* est une description observable de la tâche ; *decision_space* est l'ensemble des formes d'actions/réponses admissibles ; *required_level* est le niveau minimal de vérification ; les autres champs limitent explicitement le contexte d'exécution. Par convention, $scope_g$, $decision_space_g$ et $required_level_g$ désignent respectivement les champs *scope*, *decision_space* et *required_level* du contrat $g$. Cette projection est normative et supprime toute ambiguïté d'implémentation.

## 2.2 Contexte d'observation

Le terme $c_t$, absent de certaines versions précédentes, est désormais défini par

$$c_t = (episode_id, step, phase, contract_id, authorized_probes, \\ resource_snapshot, memory_epoch, seed, visibility_class).$$

Aucune probe ne peut lire un champ hors de *authorized_probes* et de son propre contrat. Le contexte est journalisé avec l'observation et garantit la relecture déterministe lorsque les sources externes sont déterministes.

## 2.3 Épisode et état

Une épisode est

$$\mathcal{E} = (g, E_0, s_0, m_0, c_0, \tau, B, V),$$

où $E_t$ est l'environnement observable, $s_t$ l'état interne autorisé, $m_t$ la mémoire active, $c_t$ le contexte courant, $\tau$ la trajectoire, $B$ le budget vectoriel et $V$ la famille de verificateurs autorisés.

L'état opérationnel est

$$s_t = (F_t, C_t, H_t, A_t, K_t, R_t, Q_t, \Lambda_t),$$

où $F_t$ est la fingerprint partielle, $C_t$ l'ensemble de candidats, $H_t$ l'historique compact, $A_t$ les actions admissibles, $K_t$ les certificats connus, $R_t$ les ressources restantes, $Q_t$ l'état machine et $\Lambda_t$ le ledger causal.

## 2.4 Sondes et résultat d'observation

Une sonde est

$$p = (id, version, scope, input_contract, operation, level, \\ cost, latency, preconditions, security, output_schema, \\ provenance_schema).$$

Son exécution produit

$$o_p = (raw_ref, value, view_id, view_version, Prov, \\ predicted_use, resource_delta, status).$$

$Prov$ identifie au minimum sonde, version, contexte, épisode, instant logique et références de source. Une erreur de probe ne devient jamais une observation valide par convention.

## 2.5 Empreinte partielle

Pour une vue $v$ :

$$\mathrm{Fp}_v(E) = H(\mathrm{canon}(\mathrm{View}_v(E))),$$

où $H$ est un hash d'intégrité et non une preuve sémantique. L'empreinte courante est un ensemble ordonné de faits

$$F_t = \{(v_i, hash_i, raw_i, witness_i)\}_{i=1}^{k}.$$

Les raws restent conservés ; aucune compression active ne peut supprimer sa source probatoire.

## 2.6 Poset des vues et témoins de raffinement

Les vues forment $(\mathcal{V}, \preceq)$. Une relation $v_i \preceq v_j$ est admissible seulement si un témoin enregistré $W_{ij}$ démontre sur un scope déclaré au moins une des propriétés suivantes : projection fonctionnelle exacte, implication contractuelle prouvée, ou relation empirique validée sur un jeu séparé. Un hash différent ne suffit pas à établir un raffinement.

## 2.7 Équivalence décisionnelle

Pour un état candidat $x$, le résultat d'un verificateur est un objet $VR$ défini ci-dessous. On introduit la prédicat explicite

$$\mathrm{Accept}(VR, g, a) \iff VR.status = PASS \wedge VR.level \geq required_level_g \\ \wedge VR.decision_key = a \wedge VR.scope \supseteq scope_g.$$

La décision certifiable est alors

$$\mathrm{Dec}_g(x; V) = \{a \in decision_space_g : \exists\, c, VR = Verifier_V(c, x, g),\ \mathrm{Accept}(VR, g, a)\}.$$

Pour un ensemble $X$ d'états compatibles avec $F_t$ :

$$X \approx_g a^* \iff \forall x \in X,\ \mathrm{Dec}_g(x; V) \subseteq \{a^*\} \ \wedge\ a^* \in \mathrm{Dec}_g(x; V).$$

Cette définition rend explicite que l'équivalence est relative au contrat et à la famille de verificateurs disponible. Elle ne prétend pas distinguer le monde entier.

# Chapitre 3

# Résultats de vérification et frontière de vérité

## 3.1 Interface de vérification

Un verificateur implémente conceptuellement

$$Verify : (candidate, g, context) \rightarrow VR,$$

avec

$$\begin{aligned} VR = (&status, level, decision_key, certificate_id, scope, verifier_id,\\ &verifier_version, evidence_refs, soundness_claim,\\ &external_truth_verified, deterministic_replay_ok). \end{aligned}$$

Les statuts minimum sont PASS, FAIL, INCONCLUSIVE, UNAVAILABLE. Un certificat doit indiquer sa portée et la revendication de son verificateur ; une vérification n'est jamais exportée hors de son scope déclaré.

## 3.2 Statuts de compétence et résultat

$$\begin{aligned} DOCUMENTED_INSIGHT &\rightarrow CANDIDATE_SKILL\\ &\rightarrow VERIFIED_SKILL\\ &\rightarrow EXECUTION_CONTRACT_CLOSED. \end{aligned}$$

$EXTERNAL_TRUTH_VERIFIED$ est un attribut indépendant pouvant accompagner la fermeture uniquement lorsqu'un verifier externe approprié le fournit. Sinon, DONE signifie fermeture du contrat d'exécution, jamais vérité universelle.

## 3.3 Suffisance décisionnelle

Soit $C_t$ l'ensemble des candidats actuellement compatibles. La fermeture est vraie lorsque

$$\exists a^* \quad \forall c \in C_t^{compatible}, \quad \mathrm{Dec}_g(c; V) = \{a^*\}$$

et qu'au moins un certificat terminal satisfait le niveau requis. Si $C_t$ est vide, la boucle ne peut pas conclure par vérité ; elle doit soit reconstruire des candidats, soit raffiner l'observation, soit rester UNRESOLVED.

# Chapitre 4

# Architecture ROCA-D

## 4.1 Vue d'ensemble

$$\begin{aligned} Input &\to State \to Observe_{min} \to Retrieve \to Sufficiency \\ &\to \{Probe, Bundle, Infer, Decompose, Discover\} \to Verify \\ &\to \{Act, Correct, Learn\} \to Consolidate \to SelfObserve \\ &\to Done/Unresolved. \end{aligned}$$

## 4.2 Noyaux autoritaires

1. Observation Kernel et Probe Registry ;
2. Partial Fingerprint Engine et View-Poset Witness Registry ;
3. Memory Quotient Graph et Correction Graph ;
4. Decision Sufficiency Engine ;
5. Adaptive Probe/Bundle Policy ;
6. Candidate/Composition/Decomposition Engine ;
7. Acquisition, CEGIS et Skill Registry ;
8. Independent Verification Layer ;
9. Causal Self-Observation et Meta-Acquisition ;
10. Resource Manager, Replay Ledger et Recovery Manager ;
11. External Adaptors optionnels sous contrats explicites.

# Chapitre 5

# Observation dégradée

## 5.1 Principe de moindre effort observable

À chaque étape, le contrôleur cherche d'abord une décision certifiable à partir de l'état d'observation courant. Une sonde, un bundle ou une re-observation supplémentaire n'est autorisée que si la suffisance ou la valeur marginale attendue justifie le coût. L'observation minimale n'est jamais confondue avec une vérité globale : une vue pauvre peut être suffisante pour une décision et insuffisante pour une autre.

## 5.2 Boucle de raffinement

La boucle normative est

$$V_t \to Retrieve(M, V_t) \to Sufficiency(g, C_t)$$
$$\to \begin{cases} \text{Decision/Verify}, & \text{si suffisant}, \\ \text{Policy}(Probe/Bundle/Reobserve), & \text{sinon}. \end{cases}$$

Après chaque observation, l'empreinte partielle et les témoins sont reconstruits ou complétés, puis la mémoire est consultée avant la dépense suivante. Le processus peut revenir à une vue déjà acquise lorsqu'une nouvelle condition ou un contre-exemple la rend informative.

## 5.3 Dégradation et continuité

Le comportement doit rester défini pour un budget faible comme pour un budget élevé : sous faible budget, le modèle s'abstient proprement lorsqu'aucune décision certifiable n'est disponible ; sous budget supérieur, il peut augmenter progressivement l'effort d'observation, paralléliser des sondes admissibles, apprendre une correction ou découvrir une nouvelle sonde. Cette continuité est une propriété du pipeline, pas une promesse de succès universel.

# Chapitre 6

# Compatibilité et extension du socle O81

O82_0 ne retire pas les surfaces générales acquises par O81. Toute capacité conservée doit rester subordonnée aux contrats O82 de sécurité, intégrité, budget, mémoire, vérification et provenance.

## 6.1 État runtime canonique

L'état exécutable est exposé sous forme d'un objet immuable

$$R_t = (problem, context, node, observation, deficit, memory, skills, \\ self, budget, policy, random, control).$$

dont la sérialisation canonique et le fingerprint sont déterministes. Le champ `control` porte les réservations, l'auto-observation en attente et la trajectoire de contrôle sans recevoir de vérité cible.

## 6.2 Enveloppe d'action compatible

Toute action est représentée par une enveloppe

$$A = (kind, payload, preconditions, EU, provenance),$$

où $EU$ contient au minimum le coût prévu. L'autorité de l'action est déterminée par $Q_t$ ; l'action peut être refusée par le vérificateur de contrat avant ou après exécution instrumentale, sans mutation silencieuse de l'état scientifique. Le hash d'action couvre uniquement les champs sémantiques et n'est jamais auto-référent.

## 6.3 Instruments spécialistes à interface variable

O82 accepte un portfolio d'instruments de la forme

$$z_i = (h_i, \sigma_i, u_i, c_i, \pi_i),$$

avec diagnostics de stabilité et de désaccord. Le portfolio est extensible à l'exécution ; aucun spécialiste fixe n'est requis universellement. Une décision issue d'un instrument reste soumise aux mêmes preuves de suffisance et de vérification que toute autre observation.

## 6.4 Self-health

L'auto-santé est une vue dérivée du runtime et de son historique d'actions. Elle calcule au minimum taux d'échec, incertitude et pertinence de contrôle. Elle n'a accès à aucun champ cible/label/gold/answer et ne peut seule produire une fermeture certifiante.

## 6.5 Lifecycle des compétences

Toute évolution de compétence possède identité, version sémantique, contrat, provenance et fingerprint. Les remplacements silencieux sont refusés. Les transitions sont classées INITIAL, PATCH, MINOR ou MAJOR selon la progression de version, et un root retest est requis lorsqu'une nouvelle capacité modifie la décision parent.

## 6.6 Observation progressive et re-observation

Une observation peut être réémise comme `reobserve` lorsqu'une vue déjà connue doit être confrontée à une nouvelle condition, et un `zoom` peut sélectionner une vue de résolution supérieure lorsqu'elle est déclarée admissible. Ces opérations n'annulent pas l'obligation de retrieval ni les budgets ; elles produisent une nouvelle preuve contextualisée.

## 6.7 Non-régression de potentiel

La migration O81→O82 est considérée comme une amélioration seulement si les capacités générales suivantes sont au minimum conservées : machine d'état et contrôleur autoritaires, action contracts, état runtime canonique, instruments variables, diagnostics de désaccord, auto-santé sans fuite, lifecycle/versionnement des skills, re-observation/zoom, décomposition/root retest, replay/recovery et teacher cutover. La surface CLI générale d'O81 0.3 est aussi conservée par les commandes de compatibilité `experiment`, `verify-skill`, `transfer`, `compose` et `memory-verify`. Ces commandes restent soumises aux contrats O82, à la vérification, au budget, à la provenance et au ledger. Les nouvelles capacités ROCA-D doivent pouvoir s'ajouter à cette base sans verrouiller l'espace des futures actions ou compétences.

## 6.8 Règle de contrôle

À chaque cycle, le contrôleur forme l'ensemble $\mathcal{Q}_t$ des probes, bundles, re-observations et découvertes sûres admissibles. Les opérations d'observation sont ordonnées par gouvernance :

$$Safe \succ Integrity \succ VerificationGain \succ SufficiencyGain \\ \succ ResourceEfficiency \succ Exploration \succ Novelty.$$

$Safe$ et $Integrity$ sont des filtres ; aucune action rejetée sur ces niveaux ne peut être rachetée par une utilité plus élevée ailleurs.

## 6.9 Valeur marginale normative

Il est interdit de prendre une décision de politique à partir d'une unique somme pondérée qui permettrait à un grand gain d'exploration de compenser un défaut de sécurité ou de vérification. Pour chaque $q$, on calcule le tuple

$$K(q) = (VG_q, SG_q, RE_q, EX_q, NV_q, ST_q),$$

seulement après passage des filtres sûrs et intègres. $VG$ est le gain attendu de progression de vérification ; $SG$ le gain attendu de fermeture décisionnelle ; $RE$ une efficience de ressources conforme à $B$ ; $EX$ le gain de couverture/exploration ; $NV$ la nouveauté ; $ST$ est un tie-break stable et déterministe. Une dominance de Pareto peut supprimer les options dominées sur $(gain, cost, risk, latency, uncertainty)$ avant comparaison lexicographique.

## 6.10 Budget vectoriel

$$B_t = (time, compute, memory, energy, parallelism, external_quota).$$

Chaque probe expose $\Delta B$. Un bundle sérialisé consomme la somme des coûts ; un bundle parallèle conserve le même coût total mais sa latence murale idéale est

$$latency_{parallel} = \max_i latency_i$$

si et seulement si les prerequis et la politique de sécurité autorisent l'exécution concurrente. Le contrôleur doit toujours enregistrer les deux métriques.

## 6.11 Arrêt minimal

La boucle s'arrête si et seulement si une condition de fermeture autorisée est certifiée : (i) compétence vérifiée applicable ; (ii) tous les candidats compatibles convergent vers une même décision certifiée ; (iii) verifier externe satisfait directement le contrat ; ou (iv) une action de compromis explicitement autorisée est elle-même certifiée optimale sous le budget.

## 6.12 Mémoire consultée à chaque pas

Après toute nouvelle empreinte valide $F_{t+1}$, le contrôleur exécute

$$Retrieve(F_{t+1}, g, m) \to candidates, skills, \\ corrections, probe_continuations$$

avant de demander une nouvelle probe. L'exception est limitée à un échec d'intégrité du store ; cet échec déclenche RECOVERY et est journalisé.

# Chapitre 7

# Mémoire comme géométrie de l'empreinte

## 7.1 Noeud de quotient

Un noeud mémoire est

$$N = (fingerprint_set, scope, contract_key, candidates, skills, corrections, probes, certificates, costs, coverage, provenance, status, version).$$

Le lookup exige une compatibilité de scope, contract key, visibilité et statut ; un match approximatif ne devient jamais un certificat.

## 7.2 Correction

Une correction est

$$C = (F_{fail}, g, attempt, error_kind, action_taken, probe_delta, revision, F_{corr}, verification, provenance, status).$$

Le statut d'une correction est au minimum CANDIDATE ou VERIFIED. Une correction candidate peut influencer la prochaine action ; seule sa réutilisation avec un certificat valide peut produire une fermeture.

## 7.3 Deux tempos et compression

La mémoire rapide contient épisodes, erreurs, contextes récents et propositions. La mémoire lente contient patterns consolidés, capsules vérifiées, témoins de raffinement et statistiques de politique. La consolidation est non destructive ; une vue compacte pointe vers les raws et peut être invalidée/reconstruite.

## 7.4 Invalider et oublier

ACTIVE, CANDIDATE, REVIEW_REQUIRED et REVOKED sont des statuts de mémoire. Toute modification de dépendance peut provoquer une revue et un root retest. L'oubli runtime est une éviction d'index ou de cache, jamais une suppression silencieuse d'un raw ou d'un certificat historique sous politique de rétention.

# Chapitre 8

# Moteur de candidats et de suffisance

## 8.1 Schéma d'un candidat

Un candidat est

$$c = (id, version, decision, scope, input_fingerprint, provenance, dependencies, status).$$

La décision stockée dans $c$ est une proposition ; seule la prédicat Accept produite par un verificateur autorisé peut l'élever au statut certifiable.

## 8.2 Génération de candidats

Les candidats proviennent uniquement de sources autorisées : retrieval mémoire, learner interne, composition de compétences, hypothèse générée, observation d'adaptateurs déclarés. Chaque candidat conserve une provenance complète et ne reçoit aucune crédibilité de vérité par son origine seule.

$$C_{t+1} = Filter(Generate(F_t, m_t, g_t), Obs_t, g_t, m_t).$$

## 8.3 Compatibilité

Un candidat $c$ est compatible si ses vues d'entrée sont disponibles, son scope couvre la situation, sa version est active et ses dépendances sont satisfaites. Un candidat hors scope est exclu avant comparaison de décision.

## 8.4 Fermeture et abstention

Si plusieurs décisions certifiables restent possibles, le moteur refuse l'arrêt. Si aucun candidat certifiable n'existe, il préfère l'observation/refinement ou la décomposition. En dernier ressort il produit UNRESOLVED scoped, sans déclaration d'impossibilité globale.

# Chapitre 9

# Apprentissage par erreur et correction

## 9.1 Cycle fondamental

$$Observe_{min} \to F_1 \to Retrieve \to Candidate \to Verify \to \begin{cases} Act, & \text{suffisant et certifié,} \\ Correct, & \text{échec vérifié,} \\ Refine, & \text{insuffisant,} \\ Explore, & \text{couverture inconnue.} \end{cases}$$

## 9.2 CEGIS

La boucle de correction suit

$$Hyp \to Test \to Counterexample \to Correction \to ReTest \to Holdout/OOD \to Consolidate.$$

Un contre-exemple doit rester associé à la tentative qui l'a produit. Une nouvelle preuve ne remplace pas rétroactivement la preuve précédente ; elle crée une nouvelle version de compétence/correction.

## 9.3 Primitives et composition

Les primitives sont favorisées lorsqu'elles ont un faible coût de vérification, une portée claire et une bonne réutilisabilité, mais aucun ordre fixe de complexité n'est imposé. Une compétence complexe peut être acquise directement si sa validation est suffisante ; la capsule expose alors des dépendances plus riches et une couverture plus étroite si nécessaire.

# Chapitre 10

# Capsules de compétence et acquisition

## 10.1 Schema de capsule

$$S = (id, version, contract, scope, input_views, goal, fingerprint, program, \\ tests, holdout, OOD, dependencies, corrections, coverage, \\ resource_curve, provenance, status).$$

*program* est un artefact acquis ou généré par le learner, jamais une table de réponses codée dans le noyau.

## 10.2 Cycle d'admission

$$inspect \to observe_{min} \to induce \to internal_validate \to holdout \\ \to OOD \to verify \to admit \to consolidate.$$

Le runtime FINAL ne voit que les capsules VERIFIED_SKILL compatibles avec le contrat ; les capsules expérimentales restent hors du chemin certifiant.

## 10.3 Teacher cutover

Un teacher externe peut fournir des observations autorisées pendant l'acquisition. Chaque expérience porte une politique de visibilité. Avant le cutover runtime, le système vérifie l'absence de dépendance active au teacher lorsque le mode FINAL l'interdit. Cette vérification est testée dynamiquement et statiquement.

# Chapitre 11

# Auto-observation et apprentissage de l'apprentissage

## 11.1 État auto-observable

Les champs autorisés sont action, probe, bundle, coût, budget, taille mémoire, duplication, profondeur, erreur, incertitude, désaccord instrument, variation de score, résultat de verification, gain mesuré et événement causal. Les champs de supervision interdite incluent label, gold, answer cachée, ground truth ou toute sérialisation équivalente.

## 11.2 Déclencheurs causaux

Un événement self-observation est prioritaire lorsqu'il y a stagnation, correction répétitive, désaccord significatif, surprise OOD, échec de vérification, changement de politique, admission d'une compétence, hausse inattendue du coût ou dérive de mémoire. La périodicité n'est qu'un filet de sécurité.

## 11.3 Intervention causale

Une intervention compare au minimum une trajectoire baseline et une trajectoire où seule la décision de politique ciblée change, avec les mêmes cas appariés et des ressources comparables. Le gain causal enregistré est la différence de métriques vérifiées sur les deux bras. Une méta-policy ne peut être promue qu'après ce protocole.

## 11.4 Prediction précoce

Le modèle peut prédire la prochaine observation utile, précharger une source ou réserver une ressource. Cette prédiction ne peut pas produire une certification ni supprimer une vérification nécessaire.

# Chapitre 12

# Instruments à interface variable

## 12.1 Contrat spécialiste

Un instrument peut retourner une vue variable

$$s_i : (F_t, context) \to (h_i, uncertainty, utility_estimate, \\ provenance, View_i).$$

Le master ne dépend pas d'un paquet fixe universel. Les champs peuvent être approfondis lorsqu'ils ont une valeur décisionnelle démontrée.

## 12.2 Désaccord

En cas de désaccord, le contrôleur compare la couverture, la stabilité, l'incertitude, la valeur marginale et le coût. Il peut demander une observation indépendante, une résolution plus fine ou une re-observation. Aucune préférence permanente pour un specialist n'est intégrée par convention.

## 12.3 Loss, attention, optimiseur

Loss, attention, optimiser, RAG et mécanismes de raisonnement sont des objets observables. O82 ne les interdit ni ne les érige en noyau. Leur adoption comme compétence dépend de preuves de gain sous le contrat et le budget correspondants.

# Chapitre 13

# Interfaces externes, sécurité et protocoles

## 13.1 RAG comme observateur externe

Une source documentaire est un observateur déclaré. Le résultat minimal est

$$D = (document_id, source_id, text, metadata, source_fingerprint, stale).$$

La récupération conserve provenance, coût, couverture et nouveauté. Une source devenue obsolète peut être marquée `STALE` ; cette invalidation peut déclencher la révision des dépendances mémoire, mais ne transforme jamais le texte récupéré en vérité implicite.

## 13.2 Frontière d'accès

Toute lecture externe passe par un contrat de type

$$A = (module_id, object_ids, object_types, phases, visibility)$$

et une référence de données

$$R = (object_id, dataset_id, split, lineage_hash, visibility).$$

Le refus produit une preuve d'accès refusé dans le ledger. Le firewall est indépendant de la policy de recherche et ne peut être acheté par un meilleur score d'utilité.

## 13.3 Adaptateurs black-box

Un adaptateur externe implémente

$$Query : q \to (response, request_hash, response_hash, elapsed, provenance)$$

et doit déclarer au minimum timeout, protocole d'échange, allowlist d'environnement et identification de l'adaptateur. Les réponses externes sont des observations ; elles ne contournent ni le verifier ni le contrat de décision. Un adaptateur subprocess doit utiliser un protocole borné et ne pas exposer arbitrairement son environnement au modèle.

## 13.4 Mesure vectorielle de puissance

La puissance ne se réduit pas à l'accuracy. On conserve au minimum

$$P = (skills, coverage, transfer, OOD, cost^{-1}, verification, \\ observation_utility, self_improvement).$$

Deux versions ne sont déclarées dominantes que lorsqu'aucune dimension prioritaire ne régresse sur le même protocole et qu'au moins une dimension progresse. Une régression est explicitement publiée au lieu d'être masquée par une moyenne scalaire.

## 13.5 Protocoles expérimentaux P1-P10

Les contrôles P1-P10 d'O81 sont conservés comme protocoles expérimentaux génériques : observation-only, acquisition+intervention, self-observation appariée, transfer après cutover, OOD structurel et mécaniste, contre-observateur, ablations mémoire/fingerprint/hiérarchie. Chaque protocole impose des cas appariés, une identité de cas persistante et, lorsqu'une validation externe est requise, une provenance indépendante explicitement attestée. Les protocoles mesurent les effets ; ils ne codent pas la réponse attendue.

# Chapitre 14

# Découverte de sondes

## 14.1 Apprentissage textuel généralisant et priorité des preuves

O82 peut apprendre, en plus des empreintes exactes, une compétence textuelle générique par n-grammes de caractères hachés. Cette compétence n'est liée à aucune réponse métier, mot-clé de domaine ou table de vérité : elle est entraînée uniquement sur les exemples fournis au learner.

La priorité opérationnelle est stricte :

$$ExactMatch(F, g) \succ GeneralizedText(F, g) \succ Abstention.$$

Une correspondance exacte récupérée d'une compétence `VERIFIED_SKILL` peut satisfaire le niveau de certificat déjà établi par cette compétence. Une correspondance exacte provenant d'une compétence candidate peut obtenir uniquement une preuve locale au cas courant. Une prédiction généralisée sur une empreinte jamais vue crée toujours un candidat de niveau `generative` ; elle ne peut pas fermer un contrat `exact` sans une preuve externe propre au cas.

Les seuils du learner sont appris à partir du train avec une validation interne séparée. Les données holdout et OOD ne participent jamais à l'ajustement. Lorsque le train est trop petit pour le modèle généralisant, la capacité optionnelle se désactive sans casser l'ancien chemin O82. Une erreur du modèle généralisant reste donc un état expérimental local et ne doit jamais transformer l'apprentissage de base en échec global.

Pour les expériences O82.1 et O82.c0, la configuration retenue après essais reproductibles est : dimension de hachage 16384, n-grammes de caractères 2–5, 8 époques, taux d'apprentissage 0.12, régularisation L2 $10^{-6}$. Ces paramètres font partie de la compétence apprise et restent remplaçables par expérimentation ; ils ne codent aucune solution cible.

## 14.2 Probe candidate

Une nouvelle sonde est un artefact

$$P_c = (id, version, operation, inputs, view_schema, cost, latency, scope, \\ security, provenance, induction_refs, validation_refs, \\ anti_leak, status).$$

Une composition candidate peut être

$$p_{new} = Compose(p_{i_1}, ..., p_{i_k})$$

mais sa promotion exige contrat valide, déterminisme déclaré, provenance, coût exact, sécurité, absence de redondance triviale, validation décisionnelle sur un split séparé et audit anti-target-hardcode. Une nouveauté de hash seule n'est jamais suffisante.

# Chapitre 15

# Décomposition, composition et root retest

## 15.1 Décomposition

Une décision ouverte peut être décomposée en sous-contrats $g_1, ..., g_n$ lorsque le decomposer produit une preuve de couverture des sous-objectifs et un lien parent-enfants. Pour des objets structurés, une décomposition de premier niveau peut être générée mécaniquement sans introduire de règle de domaine.

## 15.2 Root retest

Toute correction VERIFIED, nouvelle compétence admise, invalidation de dépendance ou découverte de probe pertinente peut déclencher un root retest. Le parent est rejoué avec ses nouvelles capacités ; le résultat remplace l'ancien uniquement après nouvelle vérification.

## 15.3 Composition de compétences

La composition exige compatibilité de contrat, scope, vues d'entrée, dépendances, fingerprints, verificateurs et ressources. La nouvelle capsule référence toutes les capsules sources et possède sa propre validation.

# Chapitre 16

# État machine autoritaire

$$Q = \{INIT, OBSERVE, RETRIEVE, REFINE, INFER, TEST, VERIFY, ACT, CORRECT, LEARN, MEMORIZE, SELF_OBSERVE, DISCOVER_PROBE, DECOMPOSE, RECOVER, DONE, UNRESOLVED\}.$$

Les transitions minimales sont :

| État | Condition | Transition |
|---|---|---|
| INIT | contexte valide | OBSERVE |
| OBSERVE | observation valide | RETRIEVE |
| OBSERVE | erreur/intégrité douteuse | RECOVER |
| RETRIEVE | suffisance certifiée | VERIFY |
| RETRIEVE | compétence compatible vérifiée | INFER |
| RETRIEVE | insuffisance avec probe admissible | REFINE |
| RETRIEVE | couverture inconnue | DISCOVER_PROBE ou DECOMPOSE |
| INFER | candidat produit | TEST |
| TEST | contre-exemple | CORRECT |
| TEST | validation locale OK | VERIFY |
| VERIFY | PASS requis | ACT ou LEARN |
| VERIFY | FAIL vérifié | CORRECT |
| CORRECT | correction produite | RETRIEVE |
| LEARN | capsule candidate | HOLDOUT/OOD implicites puis VERIFY |
| SELF_OBSERVE | intervention autorisée | retour vers l'état appelant avec trace causale |
| DISCOVER_PROBE | probe candidate validée | OBSERVE |
| DECOMPOSE | sous-contrats créés | OBSERVE sur enfant puis root retest |
| RECOVER | ledger/index réparé et cohérent | RETRIEVE ou OBSERVE |
| RECOVER | incohérence non réparée | UNRESOLVED scoped |
| RETRIEVE/VERIFY | aucune fermeture mais budget encore positif | REFINE/DECOMPOSE/UNRESOLVED selon preuve locale |
| ACT | contrat exécuté et enregistré | DONE |

Aucun chemin ne permet OBSERVE→DONE direct. UNRESOLVED exige un certificat scoped contenant contrat, ressources restantes, probes autorisées non exécutées, raisons d'échec/inconclusives, dernier état de preuve et absence explicite de toute revendication d'impossibilité globale.

# Chapitre 17

# Invariants d'implémentation

1. Toute observation valide possède un raw_ref et une provenance.
2. Toute fingerprint active est re-dérivable du raw.
3. Toute relation de raffinement possède un witness et un scope.
4. Tout retrieval est enregistré avant la prochaine observation, hors recovery.
5. Toute réponse certifiée possède un verifier et un certificat vérifiables.
6. Toute correction est distinguée d'une vérité et versionnée.
7. Toute compétence active possède contrat, scope, dépendances, tests et provenance.
8. Le ledger peut reconstruire le chemin décisionnel et les coûts.
9. FINAL n'active aucune compétence CANDIDATE/REVIEW_REQUIRED/REVOKED.
10. Aucun side-channel de supervision interdite ne traverse observation, metadata, hash, self-state ou logger.

# Chapitre 18

# Protocoles de validation obligatoires

| ID | Protocole | Gate |
|---|---|---|
| P01 | Observation contractuelle | scope/provenance/cout/latence/schema/sécurité |
| P02 | Observation dégradée | première action minimale ou bundle empiriquement justifié |
| P03 | Retrieval-after-observe | lookup mémoire avant nouvelle probe |
| P04 | Decision sufficiency | fermeture certifiée relative au contrat |
| P05 | Holdout | split indépendant et visibilité contrôlée |
| P06 | OOD | transformation structurale déclarée |
| P07 | Causal self-observation | effet mesuré par intervention appariée |
| P08 | Counter-observer | comparaison random/adversarial |
| P09 | Memory ablation | même dataset/skills, mémoire active variée |
| P10 | Fingerprint ablation | quotient/witness vs contrôle sans quotient |
| P11 | Hierarchy ablation | récursif vs plat sur cas/budgets identiques |
| P12 | Error-correction | erreur et correction séparées, correction re-vérifiée |
| P13 | Probe discovery | induction et validation disjoints |
| P14 | Bundle equivalence | même contrat d'information, coûts exacts |
| P15 | Scarce probes | exécution avec sous-ensemble autorisé |
| P16 | Teacher cutover | absence effective du teacher en FINAL |
| P17 | Skill identity | compatibilité de fingerprint/version/provenance |
| P18 | Resource ledger | coût/latence/mémoire reconstruisibles |
| P19 | Replay | transitions et décisions rejouables |
| P20 | No target hardcode | audit statique + sandbox dynamique |
| P21 | Recovery | corruption simulée et restauration cohérente |
| P22 | CLI/README | chaque commande documentée est exécutable et testée |

# Chapitre 19

# Contrat CLI et guide PC

## 19.1 Installation normative

Le runtime cible Python 3.10 à 3.13, Windows 10/11, Linux x86_64 et macOS, avec le noyau en bibliothèque standard uniquement. L'installation doit permettre : création d'un environnement virtuel optionnel, installation locale, exécution directe depuis le dépôt et création d'un store mémoire local sans service externe obligatoire.

## 19.2 Commandes normatives

```
o82 doctor
o82 run \\
  --problem <path> --contract <path> --budget <json> \\
  --mode EXPERIMENT --workdir <dir>
o82 observe \\
  --episode <id> --probe <id> --workdir <dir>
o82 acquire \\
  --dataset <path> --contract <path> --mode EXPERIMENT
o82 evaluate \\
  --skill <id> --split holdout \\
  --dataset <path> --contract <path>
o82 evaluate \\
  --skill <id> --split ood \\
  --dataset <path> --contract <path>
o82 replay --episode <id>
o82 memory search --fingerprint <hash>
o82 memory review
o82 probe list
o82 probe test <id>
o82 experiment --problem <path> --contract <path> --config <path>
o82 verify-skill --skill <path> --dataset <path> --split holdout --contract <path>
o82 transfer --skill <path> --input <path> --contract <path>
o82 compose --skills <skill-a> <skill-b> --contract <path>
o82 memory-verify --memory <dir>
o82 audit
```

Les arguments et noms ci-dessus sont normatifs ; toute variation doit être annoncée comme changement de version et couverte par tests/documentation.

## 19.3 Configuration

La configuration doit déclarer budget, politique de ressources, probes autorisées, mémoire, verificateurs, adaptateurs externes, seuils self-observation, seed, mode FINAL/EXPERIMENT et politique de rétention. Les chemins externes sont résolus depuis une allowlist.

## 19.4 Mode FINAL

FINAL active uniquement les compétences VERIFIED compatibles avec les contrats, interdit les teacher/adaptateurs non déclarés, conserve provenance et résultats et expose explicitement le niveau de vérité atteint.

## 19.5 Mode EXPERIMENT

EXPERIMENT autorise l'acquisition et l'évaluation de compétences candidates sans promotion automatique. Aucun artefact candidate ne peut contaminer le chemin certifiant de FINAL.

## 19.6 Recovery

Après panne, le runtime valide ledger, hash de chaîne, index mémoire, versions de capsules et certificats. Une incohérence entraîne RECOVER puis, si elle persiste, UNRESOLVED scoped ; jamais une activation optimiste.

# Chapitre 20

# Rapports et journalisation

Tout rapport de version doit conserver le template suivant :

1. Description ;
2. Architecture ;
3. Pipeline complet ;
4. Commandes d'exécution ;
5. Résultats ;
6. Interprétation des résultats ;
7. Conséquences théorique, conceptuelle et opérationnelle ;
8. Conclusions ;
9. Statut du modèle.

Chaque exécution de test et d'expérience possède un identifiant, une date logique, un seed, une commande, une configuration, un statut, des métriques, des erreurs et des hashes d'artefacts.

# Chapitre 21

# Erreurs à éviter absolument

1. Revenir à l'observation exhaustive comme contrôleur par défaut.
2. Confondre hash différent et raffinement sémantique.
3. Utiliser la nouveauté d'une empreinte comme preuve de pertinence.
4. Laisser la mémoire certifier directement une réponse non vérifiée.
5. Confondre prediction de probe et certification.
6. Remplacer une auto-observation causale par une simple télémétrie périodique.
7. Utiliser une somme pondérée capable de sacrifier sécurité/intégrité/vérification.
8. Réintroduire un paquet specialist fixe comme interface universelle.
9. Copier les paramètres d'un teacher comme substitut à l'acquisition comportementale.
10. Déclarer l'impossibilité parce qu'une ressource ou un verifier manque.
11. Supprimer les erreurs lors de la consolidation.
12. Activer une compétence sans version, provenance, scope, dépendances et tests.
13. Exposer label/gold/answer/ground truth par side-channel.
14. Composer des compétences sans vérifier leur compatibilité.
15. Faire dériver README et commandes de l'implémentation après coup ; la documentation est un contrat testé.

# Chapitre 22

# Optimisation rigoureuse

## 22.1 Mesure coût-capacité

Toute optimisation compare au minimum exactitude/fermeture, coût, latence, probes, mémoire et couverture sur le même protocole. Un gain de vitesse qui perd la vérification n'est pas retenu comme amélioration.

## 22.2 Allocation adaptative

Les statistiques de valeur marginale sont recalibrées lorsque la distribution de travail ou le portefeuille de probes change. Une policy n'est promue que sur plusieurs budgets et splits.

## 22.3 Portfolios

Plusieurs politiques peuvent coexister. La domination est toujours annoncée par rapport à un contrat, un support de test et un budget précis, jamais comme vérité universelle.

## 22.4 Pruning

Les probes/skills dominés peuvent sortir du cache actif ; preuves et raws restent récupérables.

# Chapitre 23

# Extensions

Observateur de l'observateur, apprentissage de l'optimiseur, découverte recursive de primitives, verificateurs appris, portfolios de stratégies et transfert black-box peuvent être ajoutés à condition d'utiliser les mêmes contrats, holds, OOD, intervention causale et gouvernance.

# Chapitre 24

# Checklist de release normative

| ID | Contrôle | Gate |
|---|---|---|
| G01 | provenance/dépendances | PASS/FAIL |
| G02 | observation minimale mesurable | PASS/FAIL |
| G03 | retrieval après observation | PASS/FAIL |
| G04 | certificat de suffisance relative | PASS/FAIL |
| G05 | correction memory/invalidation | PASS/FAIL |
| G06 | holdout/OOD | PASS/FAIL |
| G07 | auto-observation causale | PASS/FAIL |
| G08 | teacher cutover | PASS/FAIL |
| G09 | random/adversarial | PASS/FAIL |
| G10 | replay déterministe | PASS/FAIL |
| G11 | budget serial/bundle | PASS/FAIL |
| G12 | scarce/rich modes | PASS/FAIL |
| G13 | anti-target-hardcode | PASS/FAIL |
| G14 | README/CLI | PASS/FAIL |
| G15 | recovery corruption | PASS/FAIL |
| G16 | skill version/provenance | PASS/FAIL |
| G17 | rapports template | PASS/FAIL |

# Chapitre 25

# Clôture expérimentale O82_2 / O82_c1

## 25.1 Statut expérimental

Les versions O82_2 et O82_c1 sont les branches correctives retenues après comparaison expérimentale avec O82, O82_0 et O81 0.3 sur le corpus SMS. Elles ajoutent trois capacités sans retirer les contrats antérieurs : apprentissage par portfolio avec correction itérative, sélection de sondes guidée par la mémoire et auto-observation causale. Les résultats expérimentaux sont des preuves de capacité sur des protocoles déterminés ; ils ne constituent pas une garantie universelle de performance.

## 25.2 Portfolio de généralisation

Le learner généralisant ne se réduit plus à une seule famille de modèles. Il peut comparer des candidats génériques, notamment `char_ngram`, `char_ngram_iterative_0.50`, `rich_generic_tree` et `o81_exact_tree`. Le choix est effectué uniquement sur une partition interne de TRAIN, selon les critères de précision, couverture, transfert, stabilité, coût, complexité et ressources. Une fois le candidat sélectionné, il est réentraîné sur la totalité de TRAIN. HOLDOUT et OOD restent strictement hors du contrôle d'entraînement.

Le candidat `o81_exact_tree` conserve les sémantiques génériques O81 0.3 de représentation et d'induction afin d'empêcher une régression de potentiel. Le candidat `rich_generic_tree` ajoute des caractéristiques génériques structurées et peut dépasser le contrôle O81 lorsque les données le permettent. Aucun caractère métier SMS n'est codé dans ces mécanismes.

## 25.3 Apprentissage par répétition et correction

Soit $T$ une partition d'apprentissage et $V_i$ une validation interne. À chaque ronde $r$, un candidat $h_r$ est entraîné sur $T_r$, les erreurs internes sont identifiées, puis un nouveau pool $T_{r+1}$ est construit sans accéder à HOLDOUT/OOD. La boucle s'arrête lorsque le gain interne devient nul ou inférieur au critère d'arrêt, ou lorsque la limite de rondes est atteinte. Après sélection, le learner final est réentraîné sur tout $T$.

Cette répétition est donc une stratégie de recherche et de correction, non une poursuite jusqu'à une performance observée sur le jeu de test. Toute condition d'arrêt liée à HOLDOUT/OOD est interdite au contrôle du learner.

## 25.4 Réponse généralisée et confiance graduée

Pour une empreinte nouvelle, une compétence généralisante peut produire une *candidate response*. La sortie normative contient au minimum une décision candidate, un score de confiance gradué, la source de la décision et $certified = false$. Ce score est une confiance de décision, non une probabilité garantie, et sa calibration doit être mesurée séparément.

Une candidate généralisée ne peut pas être transformée en décision certifiée d'un contrat *exact* uniquement parce que la compétence globale possède un certificat. La preuve doit être propre au cas courant. Cette séparation permet de répondre utilement sans confondre suggestion et vérité vérifiée.

## 25.5 Observation dégradée

À chaque cycle, le retrieval mémoire précède la sélection de sonde. Une compétence compatible peut fournir un ensemble minimal $R_t$ de probes non encore observées. La politique essaie en priorité les éléments de $R_t$, puis réévalue la suffisance après observation. Si la preuve reste insuffisante, l'effort augmente progressivement. Un bundle concurrent n'est choisi que lorsque sa valeur marginale et ses contraintes de budget le justifient.

Ainsi, si une question est déjà résoluble par {`structure`, `text`}, le système n'est pas autorisé à observer systématiquement les cinq vues disponibles seulement parce qu'elles existent. Un repli exploratoire reste possible lorsque la mémoire ne fournit aucun plan suffisamment contraignant.

## 25.6 Auto-observation causale

L'auto-observation possède maintenant une voie autoritaire. Les événements de contrôle peuvent déclencher une transition `SELF_OBSERVE`, produire une observation de l'état interne, prédire une action ou une sonde suivante et réserver des ressources avant cette action. L'auto-observation désactivée est un véritable no-op pour cette programmation automatique.

Les prédictions auto-observées ne certifient jamais directement le problème utilisateur. Une intervention causale doit être comparée à un état de référence apparié et ses conséquences sont journalisées. Des cooldowns évitent qu'un déclencheur stagnant consomme tout le budget de contrôle.

## 25.7 Restauration de potentiel O81

Les surfaces générales d'O81 0.3 restent disponibles sous la gouvernance O82 : contrat d'action enrichi, état runtime canonique, instruments spécialistes, gestion des désaccords, self-health, lifecycle des compétences, re-observation et zoom, RAG observateur externe, frontière d'accès, adaptateurs black-box/subprocess, métriques de puissance, protocoles expérimentaux et commandes `experiment`, `verify-skill`, `transfer`, `compose`, `memory-verify`.

Leur conservation est intentionnelle : O82 ajoute des mécanismes de contrôle et de mémoire mais ne transforme pas l'architecture en sous-ensemble moins expressif qu'O81. Le test O81-exact reproduit sur L1 donne 94,5% sur HOLDOUT et 79,75% sur OOD, identiques au contrôle O81 0.3.

## 25.8 Langage et sortie

Les expériences SMS établissent que le modèle manipule effectivement une entrée textuelle naturelle et produit une décision symbolique textuelle, notamment dans les classes `ham` et `spam`. Les évaluations multilingues ont utilisé l'anglais, le français, l'allemand et un OOD multilingue plus large, sans fournir au learner la langue comme variable cachée.

Cette preuve ne doit pas être surinterprétée : le runtime testé n'est pas un générateur libre de prose comparable à un LLM. Sa capacité actuellement démontrée porte sur l'apprentissage et l'utilisation de représentations textuelles pour des décisions contractuelles, y compris une réponse candidate avec confiance.

## 25.9 Règles de non-régression expérimentale

Aucune version O82_2/c1 ne doit réintroduire : une réponse métier codée, une certification issue d'une confiance seule, une fuite HOLDOUT/OOD dans la sélection du learner, une observation exhaustive obligatoire lorsque la mémoire possède un plan suffisant, une auto-observation forcée lorsqu'elle est désactivée, ou la suppression d'une capacité générale héritée d'O81 sans mesure comparative.

Les rapports expérimentaux doivent distinguer explicitement : capacité de résolution, couverture, précision sélective, abstention, coût de calcul déclaré, temps mur, mémoire, provenance des données, et statut de certification.

## 25.10 Conclusion de version

O82_2 et O82_c1 constituent des extensions légitimes d'O82. Elles augmentent la capacité de généralisation et d'adaptation sans supprimer les barrières de vérification. Leur principal coût supplémentaire

est le coût d'apprentissage et de stockage associé au learner généralisant. Les scores de confiance restent des indicateurs gradués à mesurer, et non des preuves absolues.

# Chapitre 26

# Conclusion formelle

AAM O82 ROCA-D est une architecture de contrôle recursive où l'observation dégradée, la mémoire quotient des empreintes partielles, la correction d'erreurs et l'auto-observation causale sont couplées autour d'une condition de suffisance décisionnelle explicite. La politique ne choisit jamais une observation parce qu'elle est simplement «nouvelle» : elle cherche d'abord une progression de vérification et de fermeture du contrat, dans les limites de sécurité et d'intégrité, puis optimise les ressources et l'exploration.

L'ambition de généralisation est ouverte et expérimentale. Loss, attention, optimiseur, RAG, décomposition, transfert et nouvelles sondes peuvent devenir des compétences ; aucune n'est déclarée universelle avant preuve. Le modèle est donc conçu pour apprendre, corriger, vérifier, mémoriser, réutiliser et apprendre à améliorer sa propre stratégie, sans que les solutions cibles soient pré-insérées dans le noyau.